Final Project: SmartFarming System on EKS

Smart farming services on Kubernetes with secure AWS access with IAC automation using Terraform/CDK

1. Project Overview

This project builds the frontend and backend part of a SmartFarming system. The system will run on Amazon EKS, which is a managed Kubernetes service from AWS.

In this first phase, we will deploy two small Python backend services. One service will work with Amazon S3 for file storage. The other service will work with Amazon RDS PostgreSQL for farm data / IoT sensors telemetry. Other backend to **simulate IoT** system. Frontend service is a simple javascript framework UI.

The services will use IRSA, which means IAM Roles for Service Accounts. With IRSA, each pod gets temporary AWS access based on its Kubernetes Service Account. Do not store AWS access keys inside containers, YAML files, or Kubernetes secrets.

2. Project Scope

2.1 Included in this phase

- Create an EKS cluster and connect it to IAM through an OIDC provider.
- Set up IRSA so each backend service can access only the AWS resources it needs.
- Build and deploy backend and frontend services.
- Create one Kubernetes Service Account and one IAM role for each service.
- Prepare basic disaster recovery with backups, versioning, and a restore guide.
- Add cost tags and basic cost monitoring using AWS Budgets and Cost Explorer.
- Automate the infrastructure using Terraform, AWS CDK, or both.

2.2 Not included in this phase

- Full multi-region active-active deployment.
- Advanced machine learning, alerting, or telemetry processing features.

3. Backend Services

The first release contains two backend services.

Service	Purpose	AWS resource	Allowed access
storage-service	Stores and reads farming files or objects.	Amazon S3 bucket	Read, upload, delete, and list objects in one specific bucket.
farm-data-service	Stores and manages farming data. With additional IoT sensors and actuators simulator	Amazon RDS PostgreSQL	Connect to the database and run CRUD operations
Frontend-service	Visualize iot data and stream api from backend service	CDN / S3 Static website	S3 readonly

4. Architecture and Deployment

4.1 Container images

- Each service has its own Dockerfile.
- The image should use a small Python base image.
- The container should run as a non-root user.
- Images are built in CI, scanned for security issues, tagged with the Git commit ID, and pushed to Amazon ECR.
- Each service has its own ECR repository.

4.2 Kubernetes deployment

- Each service is deployed as a Kubernetes Deployment.
- Each service has a ClusterIP Service for internal access inside the cluster.
- Each service runs in its own namespace: storage and farm-data.
- Each service uses its own Kubernetes Service Account.
- Each service starts with at least 2 replicas.
- CPU and memory requests and limits are required.
- Horizontal Pod Autoscaler is configured so the services can scale when needed.

5. Secure AWS Access with IRSA

IRSA is the main security method in this design. It lets Kubernetes pods access AWS services without storing AWS keys.

The basic flow is:

1. A Kubernetes Service Account is linked to an IAM role.
2. A pod runs using that Service Account.
3. EKS gives the pod an OIDC token.
4. The AWS SDK uses that token to request temporary credentials from AWS STS.
5. The pod uses those temporary credentials to access the allowed AWS service.

The important rules are:

- The storage-service role can only access its S3 bucket.
- The farm-data-service role can only connect to the allowed RDS database user.
- The IAM trust policy must allow only the correct namespace and Service Account.
- No AWS access keys are stored in pods, manifests, or Kubernetes secrets.
- Application pods should not rely on the EKS node IAM role for application access.

6. Disaster Recovery

The initial disaster recovery target is RPO of 1 hour or less and RTO of 4 hours or less. Adjust infrastructure to comply with your DR scenario. Produce a brief document for DR runbook: technical procedure, scripts, drill plan (if possible).

6.1 Backup plan

- S3 versioning is enabled so old object versions can be recovered.
- Old S3 versions move to Glacier after 30 days and expire after 365 days.
- RDS automated backups are enabled: 7 days for development and 30 days for production.
- RDS point-in-time recovery is enabled.
- Manual RDS snapshots are taken before database schema changes.
- AWS Backup is used to manage backup plans for RDS and S3.
- Production backups are copied to another AWS Region for extra protection.

6.2 Restore testing

- Write a restore guide for S3 object recovery.
- Write a restore guide for RDS point-in-time recovery.
- Write the steps to update Kubernetes configuration after a restore.
- Run a disaster recovery drill every quarter.
- Test the restore in an isolated AWS account.
- Record how long the restore takes and compare it with the RPO and RTO targets.

7. Cost Tagging and Cost Control

Cost tracking starts from the beginning of the project. Every important AWS resource should have tags so the team can see which service and environment creates the cost.

Tag	Example	Meaning
Project	smartfarming	Groups all SmartFarming costs.
Environment	dev, staging, prod	Separates development, staging, and production costs.
Service	storage-service	Shows which microservice owns the resource.
Owner	team-platform	Shows who is responsible.
CostCenter	CC-AGRI-01	Used for internal cost allocation.

7.1 Cost control actions

- Use right-sized EKS worker nodes.
- Scale non-production workloads down when they are not needed.
- Use S3 Intelligent-Tiering for files with unknown access patterns.
- Use Graviton-based RDS instances when suitable.
- Use spot instances as this is for project only.
- Create AWS Budget alerts at 50%, 80%, and 100% of the monthly budget.
- Review AWS cost recommendations every month.

8. Milestones

Milestone	Main result	Done when
M1	EKS, OIDC, ECR, and basic IRSA are ready.	A test pod can assume its IAM role without static AWS keys.
M2	storage-service and S3 bucket are ready.	The pod can upload and download files from S3, and CloudTrail shows the correct IAM role.
M3	farm-data-service and RDS PostgreSQL are ready.	The pod can run CRUD operations using IAM authentication, without a DB password in the pod.
M4	Backup and restore are ready.	A restore test succeeds and meets the target RPO and RTO.
M5	Cost tags and budget alerts are ready.	All resources are tagged, reports are available by service, and budget alerts are tested.

9. Main Risks and Mitigation

Risk	How to reduce the risk
IRSA or OIDC is configured incorrectly.	Test IRSA with a simple pod before deploying real services.
Tags become inconsistent over time.	Use AWS Organizations Tag Policies and AWS Config rules to check tag compliance.
Backups exist but restore was never tested.	Make restore testing a required milestone before go-live.
AWS Cost during project	Use free credit wisely. Use microk8s in your local to test your kubernetes setup and servicesL: frontend, backend, and postgresql services.

10. Open Tips

- Should you exercise development env using local self-hosted service (no-cloud) and move to cloud prod once all setup well-tested?
- Agentic AI use is good path.